Curso de Git e GitHub

Silas Henrique Alves Araújo

Escola Politécnica da Universidade Federal da Bahia

Quem sou eu

Silas Henrique

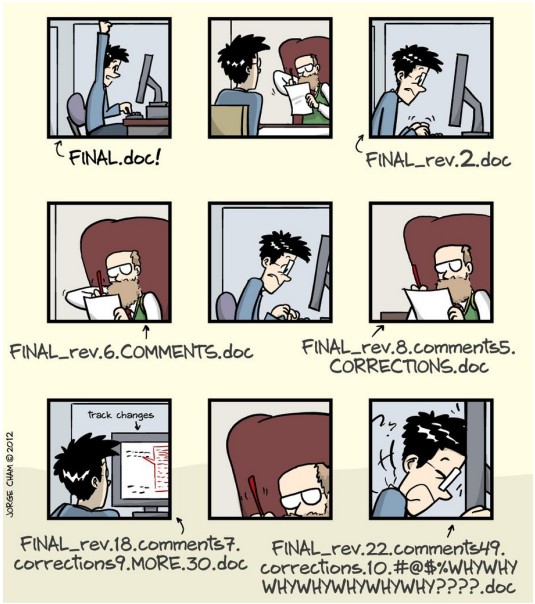
- Técnico em Eletromecânica pelo IFBA
- Graduando em Engenharia de Controle e Automação (9º semestre)
- Ex-diretor de Marketing da OPTIMUS Jr.

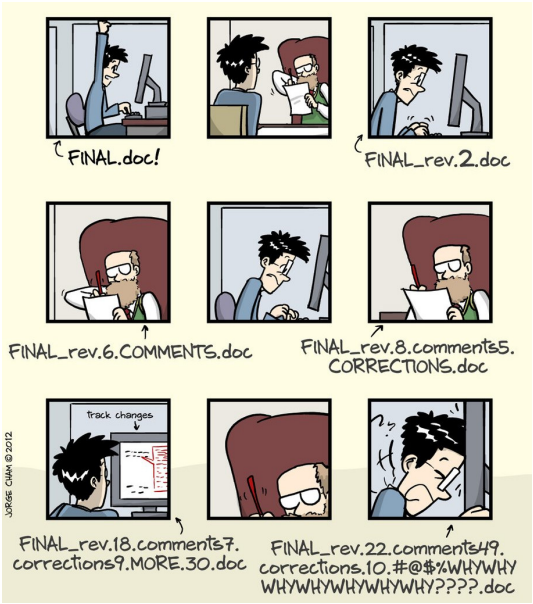
silas.eng.br



Seção 1

Por que Git?


FINAL.doc!
FINAL_rev.2.doc
FINAL_rev.6.COMMENTS.doc
FINAL_rev.8.comments5.
CORRECTIONS.doc
track changes
JORGE CHAM © 2012
FINAL_rev.18.comments7.
corrections9.MORE.30.doc
FINAL_rev.22.comments49.
corrections.10.#@$%WHYWHY
WHYWHYWHYWHYWHY????.doc



E se a gente versionasse?

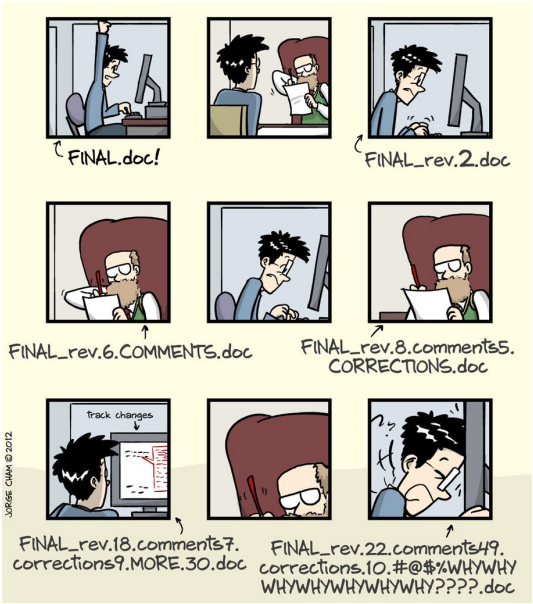
TCC-v0.1.docx

TCC-v1.0.docx

TCC-v1.1.docx

TCC-v2.0.docx

semver.org



E se a gente versionasse?

TCC-v0.1.docx

TCC-v1.0.docx

TCC-v1.1.docx

TCC-v2.0.docx

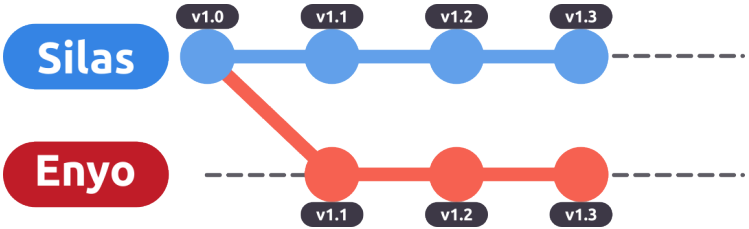
semver.org

Mas...

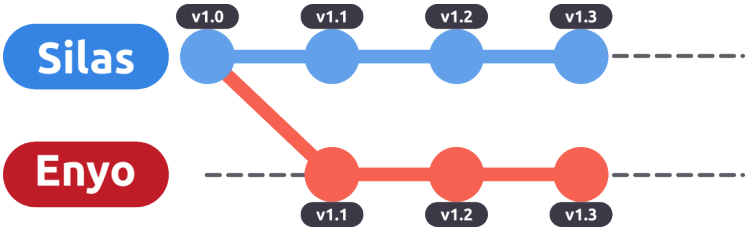
E se você quiser saber

o que mudou entre uma versão e outra?

E quando o trabalho é em grupo?



E quando o trabalho é em grupo?

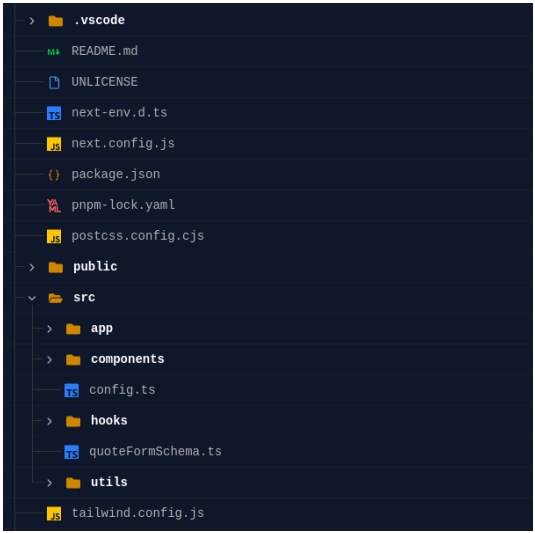


Problema

Duas pessoas partiram da mesma versão
e fizeram alterações diferentes.

O que mudou? E como juntamos tudo?

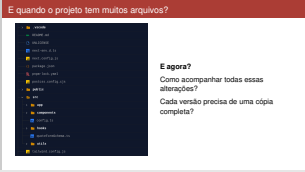
E quando o projeto tem muitos arquivos?



E agora?

Como acompanhar todas essas alterações?

Cada versão precisa de uma cópia completa?



Antigamente, era comum uma empresa ter um servidor central onde ficava o projeto inteiro. Os desenvolvedores trabalhavam nos arquivos localmente e, quando terminavam alguma alteração, colocavam os arquivos novos de volta no servidor.

Mas se uma pessoa fazia uma alteração e colocava o arquivo no servidor. Depois, outra pessoa, que estava trabalhando em uma cópia mais antiga daquele mesmo arquivo, terminava o trabalho e colocava a versão dela no servidor. A versão mais nova simplesmente sobrescrevia a anterior.

Então surgiram mecanismos de bloqueio. Quando alguém começava a trabalhar em um arquivo, ele era marcado como “em uso”. Enquanto aquela pessoa estivesse trabalhando, ninguém mais poderia alterá-lo.

Só que as pessoas esqueciam de liberar os arquivos. Alguém começava uma alteração na sexta-feira e só voltava na segunda. Enquanto isso, todo mundo que precisava daquele arquivo ficava esperando.

E, conforme os projetos cresciam, a situação ficava cada vez mais complicada.

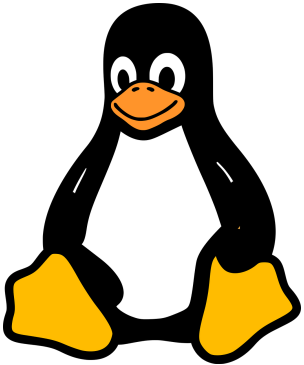
Em algumas empresas, chegou-se ao ponto de terem que contratar uma pessoa só para controlar tudo isso.

Seção 2

O Linux e o Git

O problemão: o Kernel Linux

- Mais de **43 milhões de linhas de código**, distribuídas em mais de **108 mil arquivos**.
- Mais de **13 mil desenvolvedores**.
- Várias versões do Kernel são mantidas **simultaneamente**.
- Empresas como **Google** e **Canonical (Ubuntu)** mantêm versões do Kernel com suas próprias modificações.
- Alterações precisam transitar entre diferentes versões, empresas e mantenedores, de forma **descentralizada**.
- Precisamos saber **quem escreveu cada alteração, o que foi alterado e por quê**.



O Linux e o Git

- O Linux utilizava antes a ferramenta **BitKeeper**. Era de código proprietário, mas disponibilizado gratuitamente aos desenvolvedores do Kernel.
- Em 2005, Andrew Tridgell criou o SourcePuller, uma ferramenta para interagir com repositórios BitKeeper. A BitMover considerou uma violação das condições de uso e encerrou o acesso gratuito para o Linux.
- Linus então decidiu criar o Git.



> Eu sou um desgraçado egocêntrico, então batizo todos os meus projetos com meu nome. Primeiro Linux, agora Git.
>
> — Linus Torvalds

git: pessoa cabeça-dura, argumentativa, que acha que sempre tem razão; também pode significar uma pessoa desagradável ou estúpida.

O poder do Git

O código-fonte atual do Kernel Linux ocupa:

1,7 GB

Mas o Git armazena todo o histórico do projeto, com cerca de **1,4 milhão de versões**, em apenas:

5,5 GB

Todo o histórico de desenvolvimento ocupa pouco mais de 3 vezes o tamanho do código atual!

E o GitHub?

- Em **2008**, nasceu o GitHub, uma plataforma para hospedar repositórios Git e facilitar a colaboração.
- O GitHub é um serviço proprietário.
- Existem alternativas de código aberto, como **GitLab**, **Gitea** e **Forgejo**.
- Ironicamente, em **2018**, a Microsoft comprou o GitHub por **US$ 7,5 bilhões**.
- Linux não utiliza e nunca utilizou o GitHub. O desenvolvimento utiliza seu próprio ecossistema de repositórios Git e **e-mails**.



Git

- Versionamento
- Histórico de alterações
- Autoria das alterações
- Contexto e motivo das alterações
- Comparar versões
- Mesclar alterações
- Voltar versões
- Trabalho em paralelo

GitHub

- Backup em Nuvem
- Colaboração com outras pessoas
- Code review
- Pull Requests
- Issues
- Integração contínua (CI/CD)
- Permissões e controle de acesso

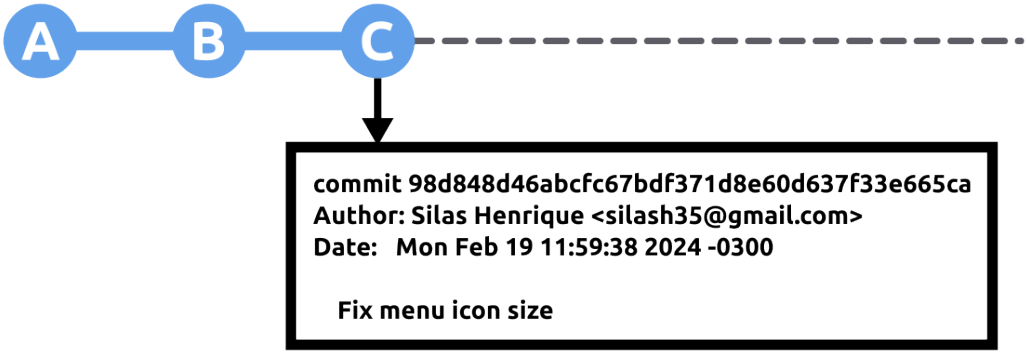
Seção 3

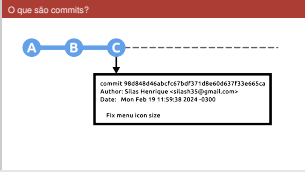
Teoria do Git

O que são commits?



O que são commits?

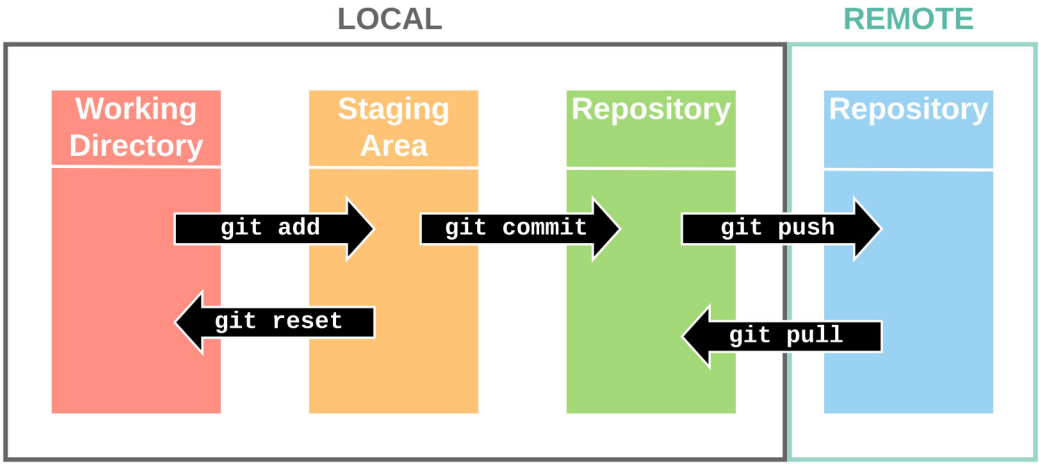


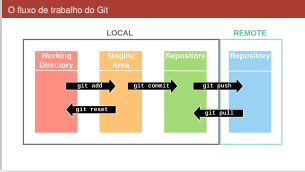


Um commit é um registro de uma alteração no histórico do projeto. Ele representa um estado do projeto naquele momento e guarda informações como quem fez a alteração, quando ela foi feita e uma mensagem descrevendo o que foi alterado. Todo commit também possui um identificador único. Esse identificador permite referenciar exatamente aquele commit no histórico do projeto.
Tá vendo? Só isso. Bem simples.
E aí, conforme criamos commits, essas relações começam a formar uma estrutura de grafo.
Mais especificamente, o histórico de commits do Git forma um grafo acíclico direcionado: cada commit aponta para um ou mais commits anteriores.

Quem achou que não ia utilizar estrutura de dados para nada?

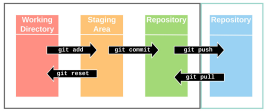
O fluxo de trabalho do Git





Vamos imaginar que estamos trabalhando em um projeto.
Primeiro, estamos simplesmente editando os arquivos. Esse é o Working Directory: o estado atual dos arquivos no nosso computador.
Agora fizemos três alterações: corrigimos um erro, adicionamos uma funcionalidade e também alteramos a documentação. Mas talvez não queiramos registrar as três coisas juntas no mesmo commit. Queremos fazer um commit apenas com a correção do erro.
É aí que entra a Staging Area. Ela funciona como uma área de preparação: colocamos nela exatamente as alterações que queremos incluir no próximo commit.
Quando fazemos o commit, essas alterações passam a fazer parte do histórico do nosso Local Repository. Esse histórico fica no nosso computador e podemos consultá-lo sem precisar de internet.

O fluxo de trabalho do Git



Depois, quando quisermos compartilhar esses commits, enviamos nosso histórico para um Remote Repository. Esse tal de Remote Repository pode ser o GitHub, o GitLab ou qualquer outra plataforma que você estiver utilizando.

Mas ele nem precisa estar na internet. Pode ser simplesmente outro repositório em outra máquina ou até outra pasta no seu próprio computador. O Git não sabe nem se importa se existe um GitHub por trás disso.

Seção 4

Prática com Git

Nosso projeto

Objetivo

Vamos escrever o cardápio de um restaurante utilizando o Git para acompanhar o processo de desenvolvimento.

Ambiente Local

No seu computador:

- Instale o Git pelo site oficial.
- Crie uma conta no GitHub.

git-scm.com **github.com**

Ambiente Online

Vamos utilizar o GitHub Codespaces.

Ele fornece um ambiente de desenvolvimento já configurado para trabalharmos com o Git.

github.com/codespaces

Primeiro passo: identificar o autor

Configurando o Git

Antes de começar, vamos informar quem está fazendo os commits.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

No **GitHub Codespaces**, essa configuração já vem preparada para você.

Primeiro passo: identificar o autor

Configurando o Git

Antes de começar, vamos informar quem está fazendo os commits.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

No **GitHub Codespaces**, essa configuração já vem preparada para você.

Uma curiosidade

Você não precisa conhecer todos os comandos do Git.

Primeiro passo: identificar o autor

Configurando o Git

Antes de começar, vamos informar quem está fazendo os commits.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

No **GitHub Codespaces**, essa configuração já vem preparada para você.

Uma curiosidade

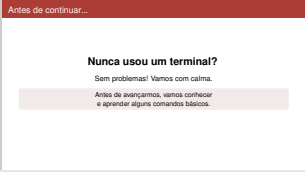
Você não precisa conhecer todos os comandos do Git.

Na verdade, existem mais de 140 comandos.

Nunca usou um terminal?

Sem problemas! Vamos com calma.

Antes de avançarmos, vamos conhecer
e aprender alguns comandos básicos.



`diff`, `patch` e hashes são conceitos importantes no Git.

Um commit pode ser representado como um patch, contendo as alterações realizadas e informações sobre o commit. Como o patch registra apenas as diferenças, não é necessário enviar uma cópia completa do arquivo, economizando espaço e facilitando o envio das alterações.

Esse patch pode ser enviado por e-mail e aplicado ao repositório de outro desenvolvedor.

Esse fluxo é utilizado no desenvolvimento do kernel Linux: os desenvolvedores enviam patches por e-mail para revisão, e os mantenedores os aplicam ao repositório.

O Git também utiliza hashes para identificar e verificar seus objetos. É uma segurança contra corrupção de dados.

Criando o projeto

Opção 1: criando localmente

Podemos começar criando um repositório na nossa máquina:

```
git init
```

Depois de criar o repositório no GitHub, adicionamos o repositório remoto:

```
git remote add origin "URL do repositório"
```

Opção 2: repositório já existente

Se o repositório já foi criado no GitHub, podemos cloná-lo:

```
git clone "URL do repositório"
```

O **GitHub Codespaces** já realiza o clone do repositório automaticamente.

Nosso primeiro commit

Criando o arquivo README.md

```
touch README.md
```

Verificando as alterações

```
git status
```

Preparando o commit

```
git add .
```

Registrando o primeiro estado do projeto

```
git commit -m "Add README.md file"
```

Subindo para o repositório remoto

```
git push
```





Escolher uma boa mensagem de commit é mais complexo do que parece: ela deve ser curta, clara e representar a intenção da mudança.
Uma convenção comum é começar com letra maiúscula e usar o modo imperativo, como `Add README.md file`, `Fix login bug` ou `Update documentation`.
A ideia é completar a frase:

> *If applied, this commit will . . .*

Em português:

> *Se aplicado, este commit irá . . .*

Não existe obrigação de usar inglês, mas minha preferência é sempre escrever os commits em inglês. Assim, não fechamos portas cedo: o projeto pode crescer, receber colaboradores internacionais ou se tornar público, e o histórico continua acessível para mais pessoas.

Consultando a história do projeto

`git log`: histórico de commits

```
git log
```

`git show`: detalhes de um commit

```
git show "commit"
```

`git diff`: diferenças entre estados

```
git diff "commit1" "commit2"
```

`git blame`: autoria das linhas

```
git blame "arquivo"
```

When you re-organize a repo and are now the git blame for all lines of code:



Arquivos úteis

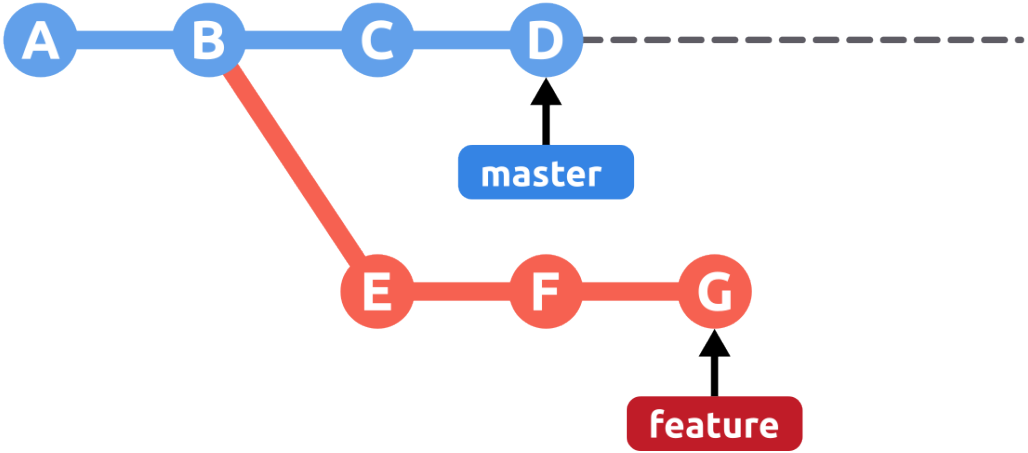
- `.gitignore`: listar arquivos que não devem ser versionados
  - `arquivo.txt`: ignora um arquivo específico
  - `*.log`: ignora todos os arquivos com determinada extensão
  - `pasta/`: ignora uma pasta inteira
  - `!arquivo.log`: não ignora esse arquivo, mesmo que uma regra anterior o ignore
- `.gitkeep`: convenção para manter diretórios vazios no repositório



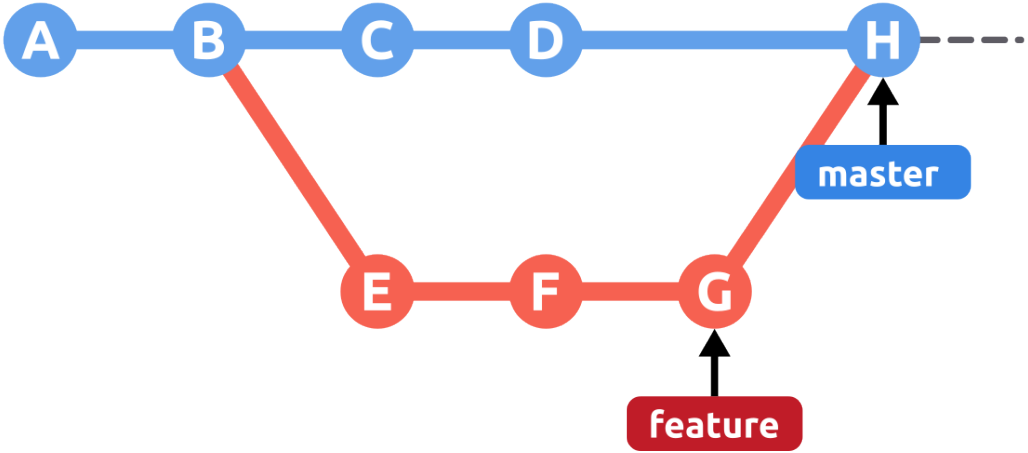
Seção 5

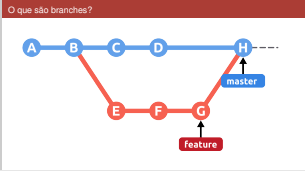
Branches e Colaboração

O que são branches?



O que são branches?





Uma branch é uma linha independente de desenvolvimento no histórico do Git.

Imagine que estamos trabalhando na versão principal do projeto e precisamos desenvolver uma funcionalidade nova. Em vez de alterar diretamente a versão principal, criamos uma branch e trabalhamos nela separadamente.

Enquanto isso, a versão principal pode continuar recebendo outras alterações. Quando terminamos a funcionalidade, podemos juntar ambas as linhas novamente mediante um merge.

O merge registra essa integração criando um novo commit que possui como ancestrais as duas linhas unidas.

Curiosidade: uma branch é apenas um ponteiro para um commit. Conforme novos commits são criados, esse ponteiro avança para acompanhar o histórico daquela linha de desenvolvimento.

Comandos básicos

`git branch` — lista as branches
`git branch "nome"` — cria uma nova branch
`git push -u origin "nome"` — envia a branch para o repositório remoto
`git branch -d "nome"` — apaga uma branch

`git checkout`

`git checkout "nome"` — muda para uma branch existente
`git checkout -b "nome"` — cria e muda de branch
`git checkout "commit"` — vai para um commit específico

`git switch` (comando mais recente)

`git switch "nome"` — mesma função do checkout para branches
`git switch -c "nome"` — cria e já muda para a nova branch

Juntando o trabalho: git merge

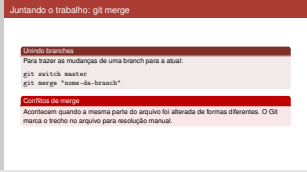
Unindo branches

Para trazer as mudanças de uma branch para a atual:

```
git switch master
git merge "nome-da-branch"
```

Conflitos de merge

Acontecem quando a mesma parte do arquivo foi alterada de formas diferentes. O Git marca o trecho no arquivo para resolução manual.



- Crie uma branch nova, por exemplo, "cardapio-sobremesas" (veja no github se criou mesmo).
- Faça um commit nessa branch, criando um arquivo de cardápio de sobremesas.
- Edite o mesmo arquivo do cardápio na master, adicionando uma sobremesa diferente, e faça commit.
- Tente dar merge da branch "cardapio-sobremesas" na master. O Git vai avisar que há conflito.
- Resolva o conflito manualmente.
- Faça um commit para salvar a resolução e finalize dando push para o repositório remoto.

Reescrevendo o histórico: push --force

`git push --force`

Sobrescreve o histórico remoto com o local, **ignorando** mudanças que outros possam ter enviado. Pode apagar trabalho de colegas!

Alternativa mais segura: --force-with-lease

```
git push --force-with-lease
```

Só sobrescreve se ninguém mais alterou o remoto desde a sua última atualização. Se alguém alterou, o push falha e avisa.

Contribuindo com outros projetos

Fork

Um **fork** cria uma cópia do repositório na sua própria conta do GitHub.

Fluxo típico

1. Faça um **fork** do repositório no GitHub
2. Clone o seu fork: `git clone "URL-do-seu-fork"`
3. Crie uma branch para sua alteração: `git switch -c "nome-da-branch"`
4. Faça as alterações, commit e push
5. Abra um **Pull Request** do seu fork para o repositório original

Pull Request

O PR permite que os mantenedores do projeto revisem suas alterações antes de incorporá-las ao projeto original.

Atividade final

Desafio: Git Wall

Vamos colocar em prática o que aprendemos fazendo uma contribuição ao projeto **GitWall**.

O que fazer

1. Faça um *fork* do projeto.
2. Faça sua contribuição seguindo as instruções do arquivo `README.md`.
3. Registre as alterações com Git e envie para o GitHub.
4. Abra um *Pull Request*.

Entrega

O *Pull Request* é a entrega da atividade para obtenção do certificado.

github.com/silash35/git-wall

Não quer aparecer no GitWall? Basta sinalizar isso no seu *Pull Request*.

Material extra

Documentação oficial

git-scm.com/learn

Tutoriais, vídeos e o livro *Pro Git*.

Git Cheat Sheet

git-scm.com/cheat-sheet

Resumo dos principais comandos e operações do Git.

Curso de Git e GitHub

Curso de Git e GitHub — YouTube

Curso de Git GitHub do CFBCursos

Pra explorar depois

- Git Rebase
- Git Stash
- Cherry-pick
- Git Revert vs Reset
- Git Flow
- Git LFS
- Git Submodules
- SSH
- Squash de commits
- Git Reflog
- Tags e Releases
- GitHub Actions
- GitHub Pages
- Issues e Projects
- Git Hooks
- Conventional Commits